29/10/2026



Restaurante con Gestión de Pedidos

Sistemas Operativos _UCOM418_81

2026

Grupo 06

William Balseca y Roberto Falquez

Fase de análisis

Contenido

Descripción.....2
Análisis.....2
Datos.....2
Conclusión de la fase de análisis.....3
Fase de planificación y diseño.....4
1. Revisión de la situación y modelo de cooperación.....4
2.Identificación del recurso compartido.....4
3. Representaciones del recurso compartido en Python.....5
4.Diseño de las clases de Python.....6
4.1Relación entre los hilos y las clases.....7
5. Identificación de la sección crítica.....8
6. Comparación de las representaciones d.....9
6.1 Representación seleccionada.....9
7. Conclusión de la fase de planificación y diseño.....10
Bibliografía.....10

Descripción

El proyecto consiste en una aplicación que representa el flujo de pedidos en la cocina de un restaurante cada pedido requiere ingredientes y debe ser asignado a un cocinero para su preparación encontramos que el problema consiste en coordinar a los cocineros cuando atienden varios pedidos al mismo tiempo, evitando que un pedido se prepare dos veces o que se utilicen ingredientes que ya fueron reservados para otro plato.

Análisis

Las unidades de ejecución participantes serán los hilos que representarán a los cocineros y el recurso compartido principal será el inventario de ingredientes además, existirá una lista de pedidos pendientes, aquí se requiere cooperación porque distintos platos pueden necesitar los mismos ingredientes y varios cocineros pueden intentar tomar pedidos al mismo tiempo concluimos que para comparar ambos modelos, se propone que la aplicación funcione en un solo equipo: con Memoria Compartida, los cocineros accederán a datos comunes; con paso de Mensajes y se comunicarán con un coordinador mediante solicitudes y respuestas.

Datos

Datos	Propósito
Pedidos pendientes	Guardar el identificador, el plato, la cantidad y el estado de cada pedido.
Inventario de ingredientes	Registrar las cantidades disponibles para preparar los platos.
Receta de cada plato	Indicar los ingredientes y las cantidades que requiere un pedido.
Resultado esperado	Obtener pedidos atendidos una sola vez, un inventario sin cantidades negativas y el registro de pedidos que no pueden prepararse por falta de ingredientes.

Tabla 1. Comparación de los modelos en la situación seleccionada

Aspecto	Memoria Compartida	Paso de Mensajes
Implementación	Los hilos de los cocineros accederán a una lista de pedidos y a un inventario comunes.	Los cocineros enviaron solicitudes a un coordinador que administraría los pedidos y el inventario.
Asignación de pedidos	Cada cocinero retiraría un pedido de la lista bajo un bloqueo, evitando que otro tome el mismo pedido.	El coordinador enviaría cada pedido a un cocinero mediante una cola de mensajes.

Uso de ingredientes	El cocinero verificará y reservará todos los ingredientes del pedido dentro del mismo bloqueo, si falta alguno, no descontará cantidades.	El cocinero solicitaba los ingredientes, el coordinador verificará y reservará el conjunto completo y respondería sí puede preparar el plato.
Coordinación	Un bloqueo o mutex protegería el inventario y otro protegería la lista de pedidos así se liberarán antes de preparar el plato.	El coordinador procesaría las solicitudes una por una y cada cocinero esperaría la autorización correspondiente a su pedido.
Ventaja	Acceso directo a los datos y una estructura sencilla para hilos dentro de una misma aplicación.	Centraliza las modificaciones y permite comunicar unidades de ejecución que funcionan en procesos o equipos distintos.
Dificultad	Sin bloqueos adecuados pueden producirse condiciones de carrera: dos hilos podrían reservar los mismos ingredientes.	Se deben definir los mensajes para administrar las colas y asociar las respuestas con los pedidos mediante identificadores.
Ejemplo	Si queda una porción de pollo, el primer cocinero obtiene el bloqueo y la reserva así el siguiente encuentra que ya no está disponible.	El coordinador recibe dos solicitudes de pollo, autoriza una y comunica al otro cocinero que ya no hay disponibilidad.

Conclusión de la fase de análisis

Para nuestro proyecto de gestión de pedidos de un restaurante, seleccionamos el modelo de **Memoria Compartida**, porque los cocineros se representarán mediante hilos que trabajan dentro de una misma aplicación y en un solo equipo, gracias a este modelo se permite acceder directamente a los pedidos y al inventario de ingredientes además utilizaremos bloqueos para evitar que dos cocineros tomen el mismo pedido o reserven los mismos ingredientes logrando así que estos bloqueos se liberen antes de preparar los platos, permitiendo que los cocineros trabajen al mismo tiempo gracias a esto consideramos que es el modelo más adecuado para coordinar los pedidos y controlar los recursos disponibles en nuestra propuesta.

Fase de planificación y diseño

1. Revisión de la situación y modelo de cooperación

Nuestro proyecto consiste en simular la gestión de pedidos dentro de la cocina de un restaurante, donde varios cocineros pueden preparar diferentes pedidos al mismo tiempo y así cada pedido necesitaran ciertos ingredientes que se encuentran disponibles en el inventario del restaurante.

Durante la fase de análisis seleccionamos el modelo de Memoria Compartida **como se puede leer previamente en el doc,** porque los cocineros estarán representados mediante hilos que trabajan dentro de una misma aplicación en Python logrando que todos puedan consultar los pedidos pendientes y el inventario de ingredientes y así lograr evitar problemas cuando varios cocineros accedan a los mismos datos, utilizaremos bloqueos que permitan controlar el acceso a los recursos compartidos.

2.Identificación del recurso compartido

El principal recurso compartido de nuestro proyecto será el **inventario de ingredientes**, donde se almacenarán las cantidades disponibles de cada ingrediente que utilizan los cocineros para preparar los platos por ejemplo, si tenemos 5 porciones de pollo disponibles y dos cocineros necesitan ese ingrediente para preparar diferentes pedidos, ambos deberán consultar el mismo inventario el implementar este recurso requiere coordinación porque, si los cocineros consultan y modifican las cantidades al mismo tiempo sin ningún control, podrían utilizar ingredientes que ya fueron reservados para otro pedido también consideramos en tener una lista de pedidos pendientes que será compartida entre los cocineros y se tiene que saber que esta lista necesita coordinación para evitar que dos cocineros seleccionen el mismo pedido.

3. Representaciones del recurso compartido en Python

Para representar el inventario de ingredientes proponemos dos alternativas:

- **Representación con tipo básico: número entero (int)**

Podemos utilizar una variable de tipo entero para representar la cantidad disponible de un ingrediente.

Ejemplo:

pollo = 5

En este caso, la variable indica que tenemos 5 porciones de pollo disponibles y cuando un cocinero utilice una porción para preparar un pedido, el valor irá disminuyendo gracias a que esta representación es fácil de utilizar, pero tiene la limitación de que solamente permite llevar el control de un ingrediente por variable, lo que dificulta administrar varios ingredientes en un solo recurso.

- **Representación con tipo complejo: diccionario (dict)**

Podemos utilizar un diccionario de Python para registrar varios ingredientes junto con sus cantidades disponibles.

Ejemplo:

```
inventario = {
   "pollo": 5,
   "arroz": 10,
   "carne": 8,
   "papas": 12
}
```

En este caso, cada ingrediente tiene una cantidad disponible y los cocineros podrán consultar el inventario para comprobar si cuentan con los ingredientes necesarios para preparar un plato creemos que esta representación permite administrar varios ingredientes dentro de una sola estructura, facilitando las consultas y las modificaciones.

4.Diseño de las clases de Python

Para nuestro proyecto proponemos utilizar cuatro clases principales, las cuales tendrán diferentes responsabilidades dentro del funcionamiento del restaurante y las desglosamos a continuación:

Clase Inventario

Responsabilidad: Administrar los ingredientes disponibles y controlar las cantidades que utilizan los cocineros.

Atributos:

- **Ingredientes:** diccionario que almacena los ingredientes y sus cantidades disponibles.
- **lock:** bloqueo que protege el inventario cuando varios cocineros intentan modificarlo.

Métodos:

- **consultar_ingredientes():** permite consultar las cantidades disponibles.
- **verificar_ingredientes():** comprueba si existen suficientes ingredientes para preparar un pedido.
- **reservar_ingredientes():** verifica y descuenta los ingredientes necesarios para un pedido, únicamente si están todos disponibles.

Clase Pedido

Responsabilidad: Representar cada pedido que realiza un cliente del restaurante.

Atributos:

- **id_pedido:** identificador del pedido.
- **plato:** nombre del plato solicitado.
- **cantidad**: cantidad solicitada.
- **estado:** indica si el pedido está pendiente, en preparación, completado o sin ingredientes suficientes.
- **receta:** ingredientes y cantidades necesarios para preparar el pedido.

Métodos:

- **consultar_estado():** permite conocer el estado actual del pedido.
- **cambiar_estado():** actualiza el estado del pedido.

Clase GestorPedidos

Responsabilidad: Administrar la lista de pedidos pendientes y controlar su asignación a los cocineros.

Atributos:

- **pedidos:** lista compartida de pedidos pendientes.
- **lock:** bloqueo que evita que dos cocineros seleccionen el mismo pedido.

Métodos:

- **agregar_pedido():** agrega un nuevo pedido a la lista.
- **obtener_pedido():** selecciona y retira un pedido pendiente para asignarlo a un cocinero.
- **consultar_pedidos():** permite consultar los pedidos pendientes.

Clase Cocinero

Responsabilidad: Representar a cada cocinero del restaurante mediante un hilo que se encargará de atender y preparar los pedidos.

Atributos:

- **nombre:** identifica al cocinero.
- **inventario:** referencia al inventario compartido.
- **gestor_pedidos:** referencia a la lista de pedidos administrada por el gestor.

Métodos:

- **run():** ejecuta las actividades del hilo del cocinero.
- **preparar_pedido():** simula la preparación del pedido cuando sus ingredientes ya fueron reservados.

4.1Relación entre los hilos y las clases

Cada cocinero será representado por un hilo utilizando la clase Cocinero, que tendrá acceso al mismo objeto de Inventario y al mismo objeto de GestorPedidos así los hilos podrán preparar pedidos al mismo tiempo, pero deberán coordinarse mediante bloqueos cuando necesiten seleccionar pedidos o reservar ingredientes del inventario, de esta manera podremos simular el trabajo de varios cocineros dentro de una misma cocina sin que se produzcan conflictos en los recursos compartidos.

5. Identificación de la sección crítica

La sección crítica de nuestro proyecto ocurre cuando varios cocineros intentan acceder y modificar los recursos compartidos al mismo tiempo.

Pensando detenidamente y viendo cómo queremos plasmar el proyecto logramos Identificamos dos operaciones que necesitan coordinación:

1. **Asignación de pedidos:** Cuando un cocinero selecciona un pedido de la lista, debemos evitar que otro cocinero tome ese mismo pedido.
2. **Reserva de ingredientes:** Cuando un cocinero verifica y descuenta ingredientes del inventario, debemos evitar que otro cocinero utilice las mismas cantidades antes de que termine la reserva.

Por ejemplo, si solamente queda una porción de pollo y dos cocineros necesitan utilizarla, ambos podrían consultar que existe una porción disponible y descontarla al mismo tiempo. Esto podría provocar que el inventario tenga cantidades incorrectas o negativas y para solucionar este problema utilizaremos *threading.Lock,* que permitirá que solamente un hilo acceda a cada sección crítica protegida a la vez, para manejar mejor esto proponemos utilizar dos bloqueos: uno para controlar la lista de pedidos y otro para proteger el inventario de ingredientes así cuando un cocinero necesite reservar ingredientes, primero tendrá que obtener el bloqueo del inventario, verificar si existen cantidades suficientes y descontar todos los ingredientes necesarios. Si falta alguno, no se descontará ninguno. Después liberará el bloqueo para que otro cocinero pueda acceder.

Los bloqueos se liberarán antes de comenzar la preparación del plato, permitiendo que varios cocineros trabajen al mismo tiempo.

6. Comparación de las representaciones del recurso compartido

Aspecto	Tipo básico (int)	Tipo complejo (dict)
Estructura	Número entero	Diccionario
Información	Cantidad de un ingrediente	Varios ingredientes y sus cantidades
Facilidad de uso	Más sencillo	Permite organizar mejor los datos
Consulta	Consulta una cantidad	Consulta cantidades por ingrediente
Modificación	Aumenta o disminuye un número	Actualiza diferentes ingredientes
Limitación	No agrupa varios ingredientes	Necesita controlar varias cantidades

6.1 Representación seleccionada

Para nuestro proyecto hemos decidido utilizar el **diccionario (dict)**, porque necesitamos administrar diferentes ingredientes que pueden ser utilizados por varios cocineros llegamos a la conclusión de que esta representación es más adecuada porque permite almacenar el nombre de cada ingrediente junto con su cantidad disponible y facilita la verificación de los ingredientes necesarios para preparar los pedidos además, utilizaremos un bloqueo para proteger el diccionario cuando los cocineros consulten y modifiquen las cantidades durante la reserva, evitando problemas de concurrencia.

7. Conclusión de la fase de planificación y diseño

En esta fase definimos cómo se organizarán las clases y los recursos compartidos de nuestro proyecto. Decidimos utilizar un diccionario para representar el inventario de ingredientes y mantener el modelo de Memoria Compartida seleccionado anteriormente.

También identificamos las secciones críticas que pueden producir problemas cuando varios cocineros trabajan al mismo tiempo y propusimos utilizar bloqueos para controlar el acceso al inventario y a los pedidos pendientes.

Con este diseño buscamos que los pedidos sean atendidos una sola vez, que el inventario mantenga cantidades correctas y que los cocineros puedan preparar diferentes platos de forma concurrente.

Bibliografía

GeeksforGeeks. (2025, 15 de julio). *Difference between shared memory model and message passing model in IPC*. https://www.geeksforgeeks.org/operating-systems/difference-between-shared-memory-model-and-message-passing-model-in-ipc/

Onsman, A. (2026, 17 de marzo). *Message passing vs shared memory process communication models*. TutorialsPoint. https://www.tutorialspoint.com/article/message-passing-vs-shared-memory-process-communication-models

Shared memory model. (s. f.). ScienceDirect Topics. Recuperado el 10 de octubre de 2026, de https://www.sciencedirect.com/topics/computer-science/shared-memory-model

Which is the best, shared memory or message passing? [Pregunta en un foro en línea]. (s. f.). Quora. https://www.quora.com/Which-is-the-best-shared-memory-or-message-passing